wwCE / CS / SE 3354 · SOFTWARE ENGINEERING · FALL 2026

Project Deliverable P1

Proposal and Requirements — Team Template

Weight 4% · Due Sunday 18 October 2026, 11:59 PM Central · Submit one PDF per team on Canvas

Field	Your entry
Project title	Simple File Explorer
Team number and section	Team 4 - 010
Professor	Sadi Evren Seker
Team members (name · net ID)	Austin Robson - axr230166, Yukai Ma - dal833810, Ashton Elliot - aee230007, George Chakiath Dani - gnc230001, Rowan Egan - rje240001, Zachary Gregory
Repository URL	https://github.com/AshtonElliott/SimpleFileExplorer/tree/main
Document version	0.10
Date	10/15/2026

0 Revision History and Contribution Record

Both tables below are graded. The contribution record carries the individual 20% of this milestone and must be consistent with your repository history.

Version	Date	Author	Summary of change
0.1	10/15/26	Austin Robson	Initial baseline for P1, added initial scope, constraints, functional requirements, quality requirements, use cases, traceability
0.2	10/16/26	Ashton Elliott	Creation of github repo, added Windows and Simple grep search tests, determined task was technically feasible.
0.3	10/17/26	Ashton Elliott	Invited George, Zac, and Yukai to github repo.
0.4	10/17/26	Yukai Ma	Add some other stakeholders, F/NF/C requirements, feasibility evidence, and come up with open questions.
0.5	10/17/26	Zac Gregory	Added project plan, roles, schedule, risks
0.6	10/17/26	Rowan Egan	Added Functional requirements and constraints
0.7	10/18/26	George Chakiath Dani	Added Context Diagram as well as finding the rest of the stakeholders and checked with the AI on how they can be affected by the app.
0.8	10/17/26	Ashton Elliott	Reorganized GitHub repo and provided description of the technical feasibility test.
0.9	10/18/26	George Chakiath Dani	Added the 1 more quality requirement and finally added the description at the end of the Context Diagram.
0.10	10/18/26	Austin Robson	Added more quality requirements, reviewed and finalized content before submission

0.1 Individual contribution record

Team member	What this person produced for P1	Evidence in the repository
Yukai Ma	Add several stakeholders. Add constraints and constraint requirements. Complete what breaks when a dependency doesn't work. Add several functional requirements and adjust the MoSCoW. Come up with open questions. Add quality requirements.	Google doc history
Zac Gregory	Project plan: Assigned team roles, plan milestones for P1 up to P2, assess general project development risks.	Google doc history
Austin Robson	Drafted initial project proposal and requirements, Added use case diagram, added/edited/verified further requirements/constraints/use cases	Google drive/doc ownership/history
Ashton Elliott	Creation of GitHub repo	In GitHub README.md

Team member	What this person produced for P1	Evidence in the repository
	Conducted Windows Search Engine Test Conducted Linux Grep Test for Technical Feasibility Determined technical feasibility.	In GitHub “RequirementsEvidence” folder under “GREP Technical Feasibility Test”
Rowan Egan	Completed functional and constraint requirements	Google doc history
George Chakiath Dani	Finished Context diagram showcasing one box for our system, every external actor and external system around it, and a labelled line for each interface in Section 2.1. I also added a description that describes the context diagram and how it represents the app with the external actors connecting the internal systems within the boundary. An AI was also added in section 4.5 describing my reasons for why we could use AI within our SRS document. Filled in the rest of the stakeholders and talked with the rest of the group to see what they feel about the addition and checked with the AI on how they can be affected by the app. Added 1 more quality requirement with a measurable threshold, appropriate priority, and a clear method of verification with the help of an AI to identify the quality requirement suitable for the app.	Software Engineering Lectures/Gemini/ChatGPT

1 Introduction

1.1 Purpose of this document and of the system

Type here.

Simple File Explorer (SFE) is a slimmed down reimagining of the stock file explorer on Windows, reducing lag and long loading times by leveraging linux commands in a simple GUI.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
GUI	Drive manager
File probing	Settings
Quick access sidebar	Networking

In scope — we will build this	Out of scope — we will not, and why
Searching	File movement

Why the exclusions were excluded:

Drive management and file movement outside of the dedicated operating system methods is tricky and potentially dangerous, and therefore out of scope for this file reader app. Network connectivity introduces vulnerabilities, and there are no options to necessitate a settings menu.

1.3 Definitions, acronyms and abbreviations

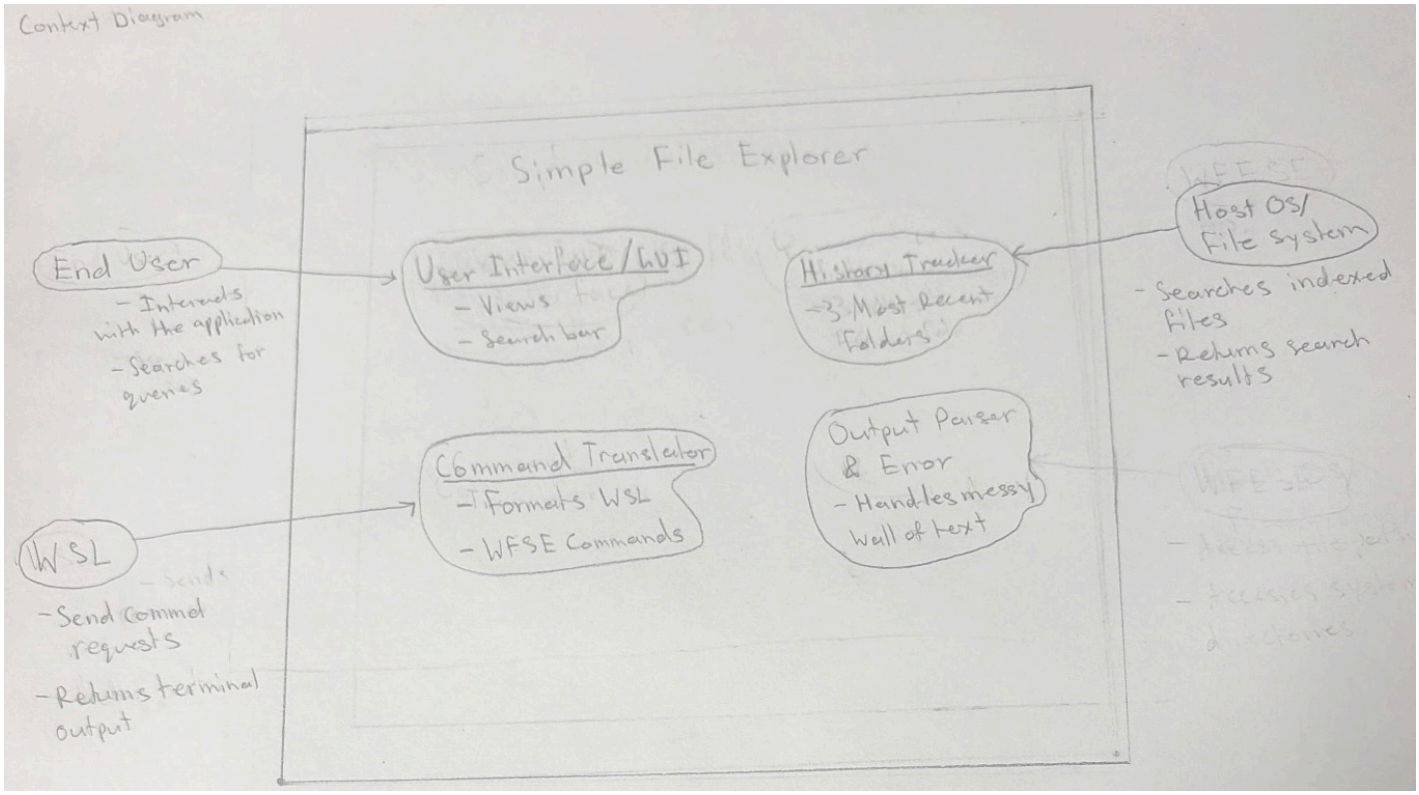
Term or abbreviation	Meaning as used in this project
SFE	Simplified File Explorer; Name of app
WSL	Windows Subsystem Linux; App relies on it for linux use
WFESE	Windows File Explorer Search Engine; Windows File Searcher that comes with all windows computers.

1.4 References

1. Windows File Explorer (WFE
2. Windows Subsystem Linux (WSL)
3. Java

2 Overall Description

2.1 Product perspective and context



Description: The Simple File Explorer system allows users to search for files through a user interface. The system is responsible for handling user requests, translating them into WSL commands, processing search results, and tracking search history. The End User, Windows Subsystem for Linux (WSL), and Host OS/File System are outside the system boundary because they are not being developed as part of the project. Instead, the Simple File Explorer communicates with these external systems to execute searches and retrieve file information. This boundary defines what our project is responsible for building and what it relies on externally.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
End users	Downloads and uses the app	A file searcher that takes less than 10 minutes to find the desired file from 300,000 files with the right path.	We ran a test on Window's File Explorer search engine to see how long it would take to find a file with the keyword "BERT". It took 11 minutes and 9 seconds to find a file with the keyword "BERT", but it was a cached path and not the actual file path.
Software Testers	Tests functionality and performance of app	The app must return the accurate file paths and accurately find files within the required time.	Based on the Windows File Explorer test, which took 11 minutes and 9 seconds and returned a cached path.
Regulator	Check if the app is legal	Need the app to obey the law (e.p protect private information)	To check all related law requirements before programming
Software Developers	Design, build and maintain the app	Develop a reliable and efficient file search app that demonstrates fast searching times for each file searched	Based on responsibilities of the development team and our group members
Maintenance Team	Fixes Bugs and updates the app after release	Code that can be easy to maintain, along with useful messages and comments for future debugging.	Based on the need to keep the app working after development

2.3 User characteristics

The users are individuals who want to find files within their computer as soon as possible. They should know how to use a search engine and that Windows has a file search engine. What cannot be assumed that they know is the fact that said Windows file search engine is inefficient as it takes 11 minutes and 9 seconds to find a single result.

2.4 Constraints

- Must run locally
- Must follow the law to protect users' information and documents.
- Must use Linux commands to make system calls

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	WSL installed	The system will not be able to execute Linux commands, and the program will fail.
A2	Java installed	The system will not be able to execute the program.
A3	Java is able to interface with linux commands	Java will not be usable for completing this project

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Yes	A small test was conducted within a Linux Virtual Machine where the keyword “BERT” was searched throughout the entire Virtual Machine with grep.
Operational	Yes	From the test, the system does increase the loading speed a lot. The end users who really want fast access to the file will choose to use it.
Schedule	Yes	Through the team discussion, the estimated time is within the deadline.
Economic / resource	Yes	Because this is a small app with no paid external dependencies, the economic cost will be little to none

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The file information displayed in the GUI shall be accurate to its actual data	MUST	Devs	Verify displayed information with what is shown by command line tools
FR-02	Folders shall be navigable via mouse or keyboard actions (double click, arrows, enter, backspace, etc)	MUST	Devs	GUI navigates through directories and displays new contents without error
FR-03	Users shall search for strings in directory/file names/contents	MUST	Devs	File searches in the GUI are accurate with results from command line tools
FR-04	Drives, commonly used folders, and folder history shall be displayed in a sidebar	MUST	Devs	SFE includes a sidebar for quick access to drives, common folders (such as Documents, Downloads, etc), and recent folders.
FR-05	Errors shall be reported within the GUI	SHOULD	Devs	Test errors display correctly when triggered and cancels the action
FR-06	Images shall display with thumbnails	COULD	Devs	Images have a smaller version (a thumbnail) generated and cached to display alongside their information

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-07	Users shall be able to create, delete, and rename files.	WON’T	Devs	Users are able to create/delete/rename files within the GUI
FR-08	Users shall be able to move files from one directory to another.	WON’T	Devs	Files are able to be transferred to other directories, by copy-paste or drag ‘n’ drop, without loss of file/directory sanity.

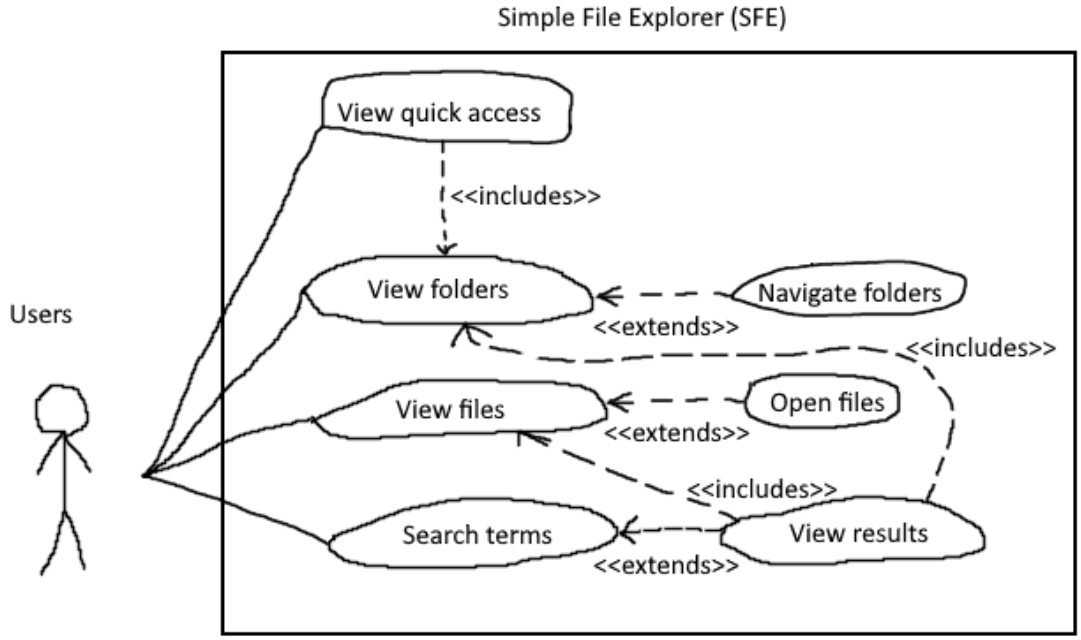
3.2 Quality requirements

ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	Searches must complete within 10 minutes of their initialization	MUST	Devs	A tester can confirm this by tracking load times while searching files.
QR-02	Entering new folders should load their new content within 5 minutes of their opening	MUST	Devs	A tester can check whether the folder and documents are displayed in time
QR-03	Search results shall contain 20 results before needing to scroll	SHOULD	Devs	Search results containing >20 entries will create a scroll bar in the window
QR-04	The app shall open within 5 minutes of its starting	SHOULD	Devs	Start times for the app will be measured to find if it successfully boots in under 5 minutes
QR-05	Errors shall be logged within 1 minute of their occurrence	SHOULD	Maintenance Team	A tester can trigger 10 different errors and verify that each is recorded in the system within 10 seconds.

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	Must run locally	Devs	Running with or through a third-party service creates unnecessary load times and opens up vulnerabilities
CON-02	The system doesn’t write, move or delete any user file	Devs	Read-only access removes the risk of data loss
CON-03	Must use Linux commands to make system calls	Devs	Use Linux commands that can achieve fast access to the file in memory

3.4 Initial use cases



Expanded use case — copy this block for each of your two

Field	Content
Identifier and name	UC-02 - View folders
Primary actor	Users
Supporting actors	N/A
Preconditions — what must be true before it starts	- Linux subsystem must be loaded on the user's system - App must have started without errors - App must have access to the user's main memory and the Linux subsystem
Trigger — what starts it	- Double-clicking on folders
Main success scenario — numbered steps	1. User double-clicks a folder 2. App executes a Linux command and uses results to build a display of the new folder's contents 3. GUI displays new contents
Alternative and exception flows	- Inaccessible folders (user permissions) result in errors and no change to currently displayed folder
Postconditions — what is true when it succeeds	- Linux command is able to be executed without error (syntax, permissions, etc) - GUI navigates to new folder correctly - GUI displays new folder contents accurately
Requirements realised	FR-02

Expanded use case — copy this block for each of your two

Field	Content
Identifier and name	UC-01 - View files
Primary actor	Users
Supporting actors	N/A
Preconditions — what must be true before it starts	- Current directory must have opened without error - Current directory must be rendered accurately (the gui is actually in the correct folder) - App must have permission to view files in the directory
Trigger — what starts it	- Having opened a folder
Main success scenario — numbered steps	1. Files in current directory are displayed with relevant information (time, size, type, etc)
Alternative and exception flows	- App lacks permission to view folder contents, resulting in files not being displayed
Postconditions — what is true when it succeeds	- Files are displayed with information accurate to what is shown outside of the app (running the same commands in the terminal shows the same information)
Requirements realised	FR- 01

3.5 Requirements to use case traceability

Requirement ID	Use case(s) that realise it	If blank — why, and what you will do
FR-01	UC-01 - View files	
FR-02	UC-02 - View folders	
FR-03	UC-03 - Search terms	
FR-04	UC-04 - View quick access	
FR-05		Visual error reporting is a consideration and hasn't been made into a full use case, therefore it is considered a COULD priority requirement in the initial project.
FR-06	UC-01 - View files	
FR-07		File manipulation is slated as out of scope for the current project version, therefore it will not have a use case and is set to WON'T in requirement priority
FR-08		Same as above

What the matrix revealed, and what you changed because of it:

Decisions had to be made on FR priority based on what use cases it matches up with. This let us clear up what is and isn't important as a Functional Requirement for this version of the project.

4 Project Plan

4.1 Team roles and working agreement

Team member	Role for P1 to P2	How the team makes a decision when it disagrees
Yukai Ma	Questioner, Analyzer (stakeholder liaison, requirement summary)	We discuss and value each one's opinion, and we will debate and vote for the final solution.
Zac Gregory	Risk manager, plan tracker	
Austin Robson	Project manager	Project baseline will be considered in regards to the topic at hand, and explanation will be offered to explain how it fits/does not fit to allow further discussion and reach an understanding.
George Chakiath Dani	Use case and modeling lead	We identify the actors interacting with the system and model how they interact with the application through use case diagrams and scenarios.
Ashton Elliott	Feasibility and performance engineer	
Rowan Egan	Requirements QA	

4.2 Plan to P2

Date	What will be finished	Owner
10/18	P1 baseline	Austin Robson
10/20	P1 feedback review	Yukai Ma
10/25	Requirements revised: QR-02 - QR-04, SRS updated to v0.2	Rowan Egan
10/28	Technical stack decided, draft architecture written	Rowan Egan
11/1	Class, sequence, and state diagram drafts	George Chakiath Dani
11/4	Risk register and open questions updated	Zac Gregory
11/6	Full-team P2 document review	Zac Gregory
11/8	P2 submission	Austin Robson

4.3 Risks

#	Risk	Likelihood	Impact	Mitigation — action and owner
R1	Team member(s) unable to get WSL and Java working on their machine.	Low	Low	Ashton Elliott checks with every member and has installed instructions in the README. Those still with problems will be helped 1 on 1.
R2	Calling Linux commands from Java through WSL turns out slower than anticipated.	Medium	High	Ashton Elliott runs a grep search through WSL against a folder and records the times and posts them to the repo before the architecture is finalized on 10/28. Missing the target will require revisiting QR-01.

#	Risk	Likelihood	Impact	Mitigation — action and owner
R3	External commitments like other classes or jobs leaving a person(s) unable to deliver their work by the deadlines.	Medium	High	Zac Gregory checks on everyone's individual progress on days 10/28 and 11/1, names backups for each task, and reassigns work if something is more than 2 days behind schedule.
R4	Requirements change informally.	Medium	Medium	Rowan Egan keeps every post-baseline change as a logged entry of the revision history. The same change must be reflected in the traceability matrix which is checked by Zac Gregory on 11/6 review.
R5	The design is built on an unresolved premise: Q1 and Q2 are open.	Medium	High	Yukai Ma gets answers or a team decision recorded in section 5 by 10/20, and any change to the functional requirements or constraints goes through the revision history.

4.4 Repository and tooling

https://github.com/AshtonElliott/SimpleFileExplorer/tree/main

4.5 Declaration of AI tool use

Tool	What it was used for	Who used it	How the output was checked
Gemini	Understanding how context diagrams work and how to identify the external actors and internal system and the connection in between within the system boundary	George Chakiath Dani	The output was checked by recognizing the different external actors that were outside of the system boundary and the internal systems which were inside the boundary along with our desired system. Each external actor then connected the internal systems with a brief set of bullet points on how each actor was utilizing each system.
ChatGPT	Researching more stakeholders and seeing how they relate to the system, what their needs are, and how they may impact the app's development.	George Chakiath Dani	The output was checked to ensure accuracy, completeness as well as consistency with the project requirements. It was also reviewed by the team members to ensure that all relevant stakeholders and their needs were properly identified.
ChatGPT	Finding out more quality requirements to add, as well as how much should be included in the project	George Chakiath Dani	The output was checked by reviewing the quality requirements to ensure they were clear, measurable, relevant to the project, and assigned appropriate priorities.

5 Open Questions

#	The question	Who can answer it	Needed by	What you assumed in the meantime
Q1	As this file explorer needs to run Linux in Windows, why do we just use Windows local commands?	Austin Robson	10/18/2026	The user will only be able to use the app if linux subsystem is installed
Q2	Do we need to add a tutorial to teach users to download WSL before using this file explorer?	Austin Robson	10/18/2026	Users will have to learn how to install WSL themselves

6 Baseline Declaration

We confirm that the requirements in section 3 represent our agreed baseline as of the submission date, and that every team member has read this document in full.

Team member	Signature or typed name	Date
Austin Robson	Austin Robson	10/18/2026
George Chakiath Dani	George Chakiath Dani	10/16/2026
Yukai Ma	Yukai Ma	10/18/2026
Ashton Elliott	*Ashton Elliott*	10/18/2026
Rowan Egan	*Rowan Egan*	10/18/2026

How P1 Is Marked

P1 is worth 4% of the course grade. Following the syllabus, 80 points are for the shared team artifact and 20 for individual contribution evidence.

Criterion	Points	What full marks look like
Problem, scope and context (1.1, 1.2, 2.1)	10	The problem is a problem somebody actually has. The out-of-scope column is filled and the exclusions are justified. The context diagram's boundary matches what sections 2 and 3 claim.
Stakeholders and elicitation (2.2, 2.3)	10	Stakeholders include at least one who will never use the system. Every need has a stated basis, and assumptions are labelled as assumptions rather than disguised as findings.
Requirements quality (3.1, 3.2, 3.3)	25	Every requirement is one testable “shall” sentence with an identifier, a priority, a source and an acceptance criterion. Quality requirements carry a threshold, a condition and a method. Constraints are distinguished from quality requirements.
Use cases and traceability (3.4, 3.5)	20	Use cases are at actor-goal level, not steps. Two are written out in full, including alternative flows. The matrix is complete and the team says what the gaps mean and what they did about them.
Feasibility and plan (2.6, 4)	10	Each feasibility verdict is backed by evidence a reader can check. The plan has dates and owners. Risks are future events with actionable mitigations.
Open questions and baseline (5, 6)	5	Real open questions, with an owner and a date, and an honest statement of what was assumed in the meantime.
Document quality	—	Guidance boxes removed, sections in order, within the length budget, readable. Marks are deducted here, not awarded.
Individual contribution evidence (0.1)	20	Every member's contribution is distinguishable and traceable to something in the repository a reader can open.